Fleetix

Route Optimisation Solution for Employee Transportation

[Project Release 1 by Team 28]

01

Problem Description

“If I had an hour to solve a problem, I'd spend 55 minutes thinking about the problem and five minutes thinking about solutions.” - Albert Einstein

Problem Statement

Client :

Efficiently transport employees with multiple pickup and drop-off locations with added constraints (like waiting time, vehicle capacity and maximum allowed delays)

Us :

Build a route optimization tool for employee transportation, addressing scenarios involving multiple pickups and drop-offs and managing constraints as prescribed by the client's requirements

Sub-Problems

Breaking down the problem, we get the following major implementation steps :

1. **Geocoding** : Extracting geo-coordinates of various locations, from their textual addresses.

2. **Routing** : Finding the optimal path connecting the N points.

3. **Fleet Management** : Managing the multiple vehicles, routes and employees; under constraints such as vehicle capacities, time preferences, etc.

02

Our Approach

“The best transportation system is the one you don’t notice - because it never disrupts work.”

SDLC Methodology

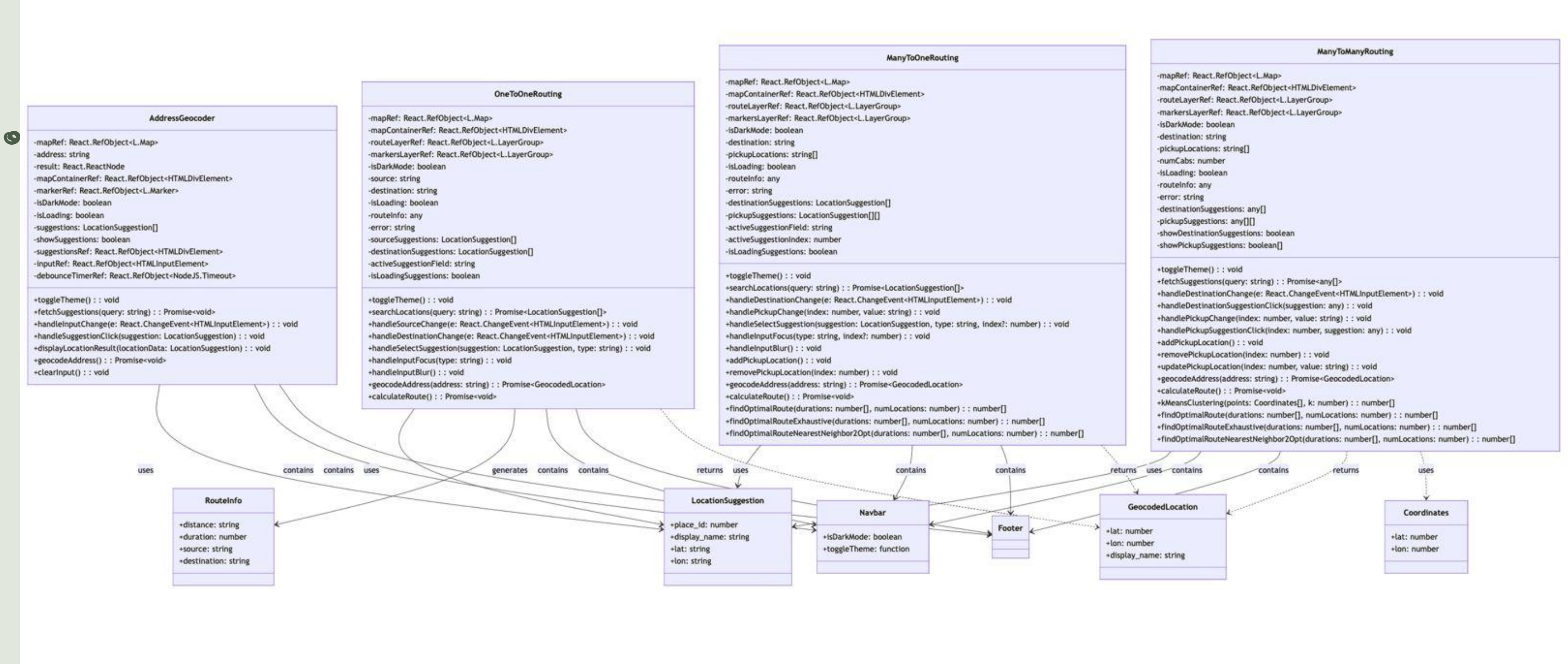
We have tried following the **Agile (SCRUM) Methodology** throughout our project implementation. Some key ideas that we have incorporated are as follows :

- Conduct daily stand-up meetings for progress updates.
- Maintained sprint cycles (deadlines) with clear deliverables.
- Iterative cycles enabling quick adaptation to changing requirements.
- Continuous feedback loop allowing for quality improvements throughout development.

Apart from this, we have also tried maintaining **detailed documentation** of every design choice, implementation method and architectural decisions.

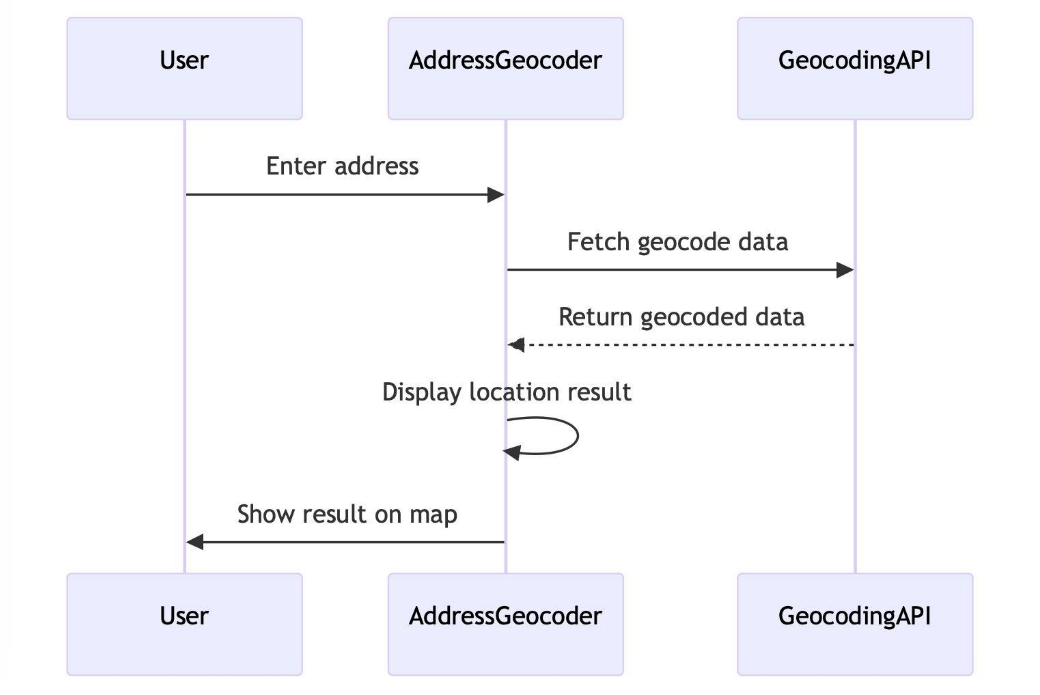
Project Flow

Class Diagram



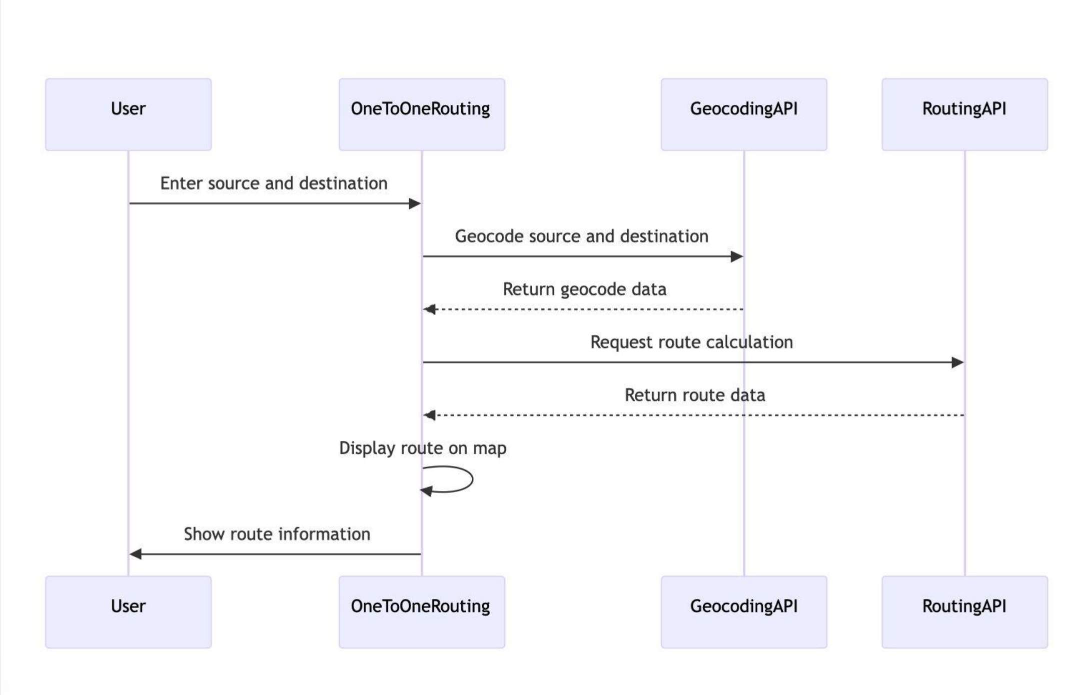
Project Flow

Sequence Diagram - Geocoding



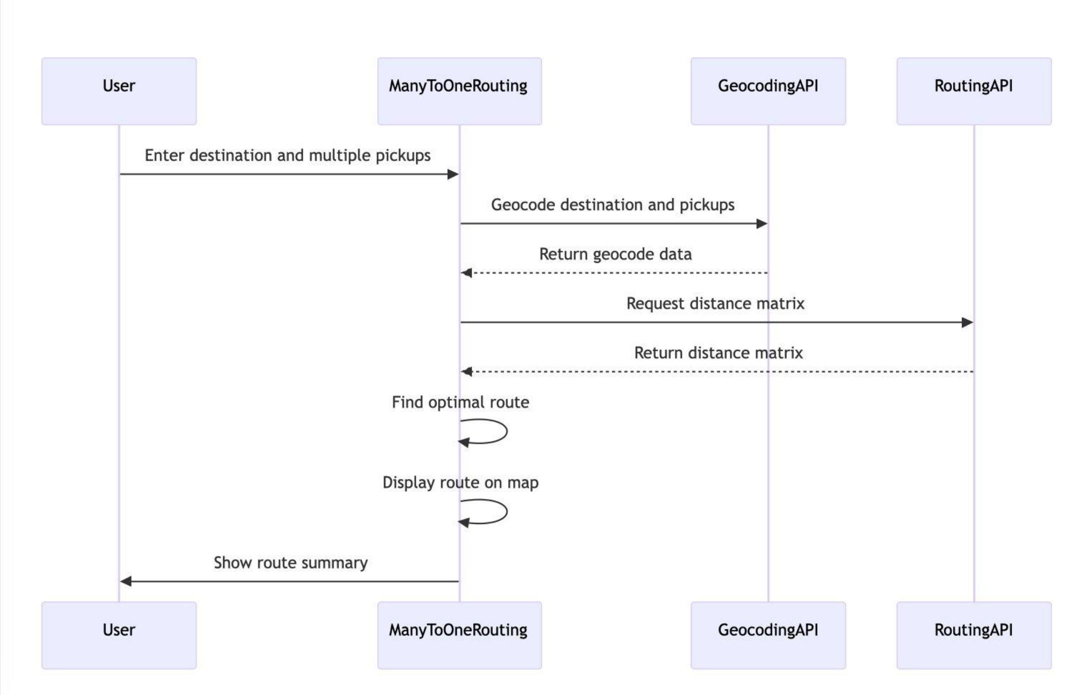
Project Flow

Sequence Diagram - One-to-One Routing



Project Flow

Sequence Diagram - Many-to-One Routing



Tech Stack

For the current implementation of the project, we have used the following frameworks and libraries

User Interface (Frontend)

- Next.js
- Typescript
- Tailwind CSS

Backend

- PostgreSQL (Not Integrated with Frontend yet)

Routing

- OSM (OpenStreetMap) + OSRM (Open Source Routing Machine)
- More details on next slide

Deployment

- Vercel : https://fleetix-team28.vercel.app

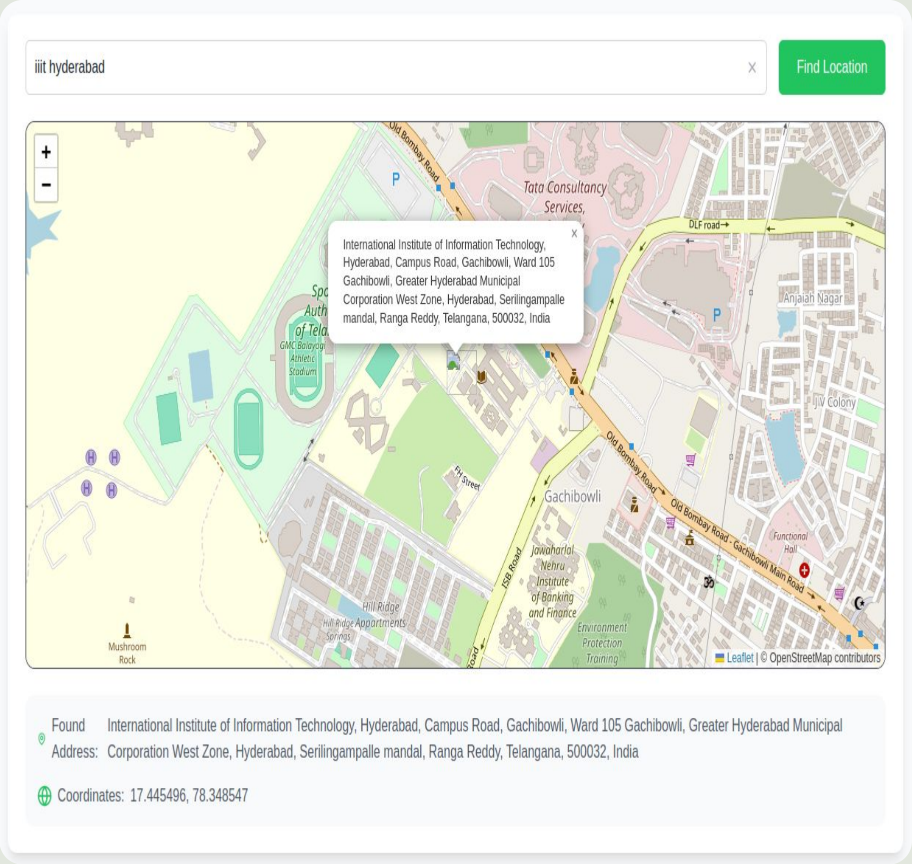
Resource Library

Based on our preliminary market research and pre-existing solutions, we planned on going ahead with **OSM (Open Source Map)** and **Open Source Routing Machine (OSRM)** for our implementation.

Some of the most significant reasons were :

1. **Data Ownership** : Based on the locality and density of the road network required, one can choose their own road network data.
2. **Community Support** : A strong contributor community ensures ongoing support, development and improvement.
3. **Free and Open Source** : The client favoured an open-source solution, over a proprietary one.
4. **High-Volume Route Optimization** : Batch routing (multi-vehicle, multi-stop planning), which Google Maps API charges extra for via the Distance Matrix API.

Address Geocoding

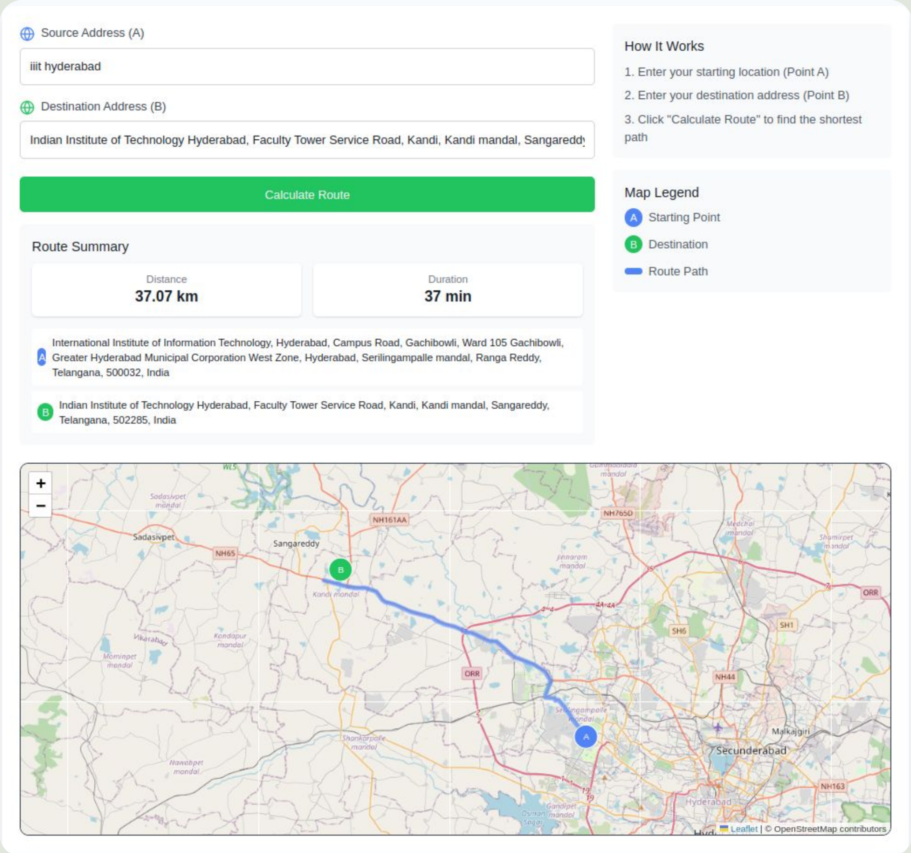


- Process of **converting textual addresses into geographic coordinates** (latitude and longitude).

- These coordinates can then be processed by the Open Source Routing Machine (OSRM) to generate optimized routes and travel time estimates.

- Allows precise location mapping, making it easier to integrate address-based data into digital applications like mapping, navigation, and route optimisation.

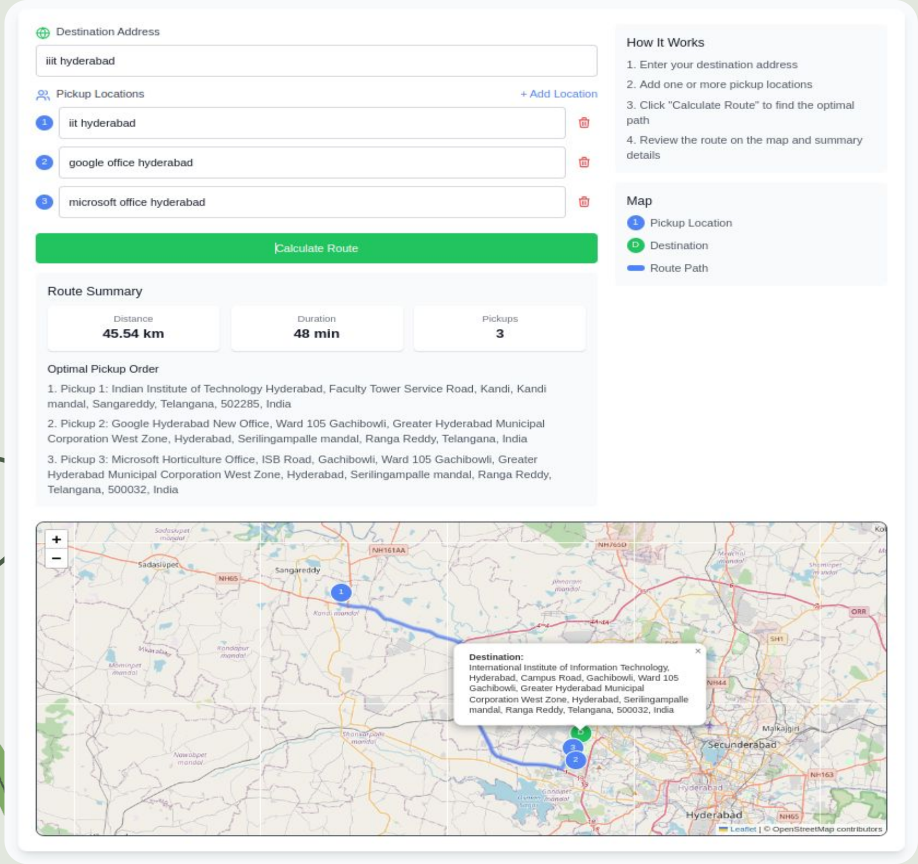
Routing and Optimisation



Can be categorised into 3 broad categories :

- **One-to-One** : Singular route from one source to one destination
- **Many-to-One** : Singular route from a source to destination; with multiple pick-up points in between
- **Many-to-Many** : Multiple routes (fixed number) covering a set of points. Each point falls under one of the route; but no route necessarily covers all points.

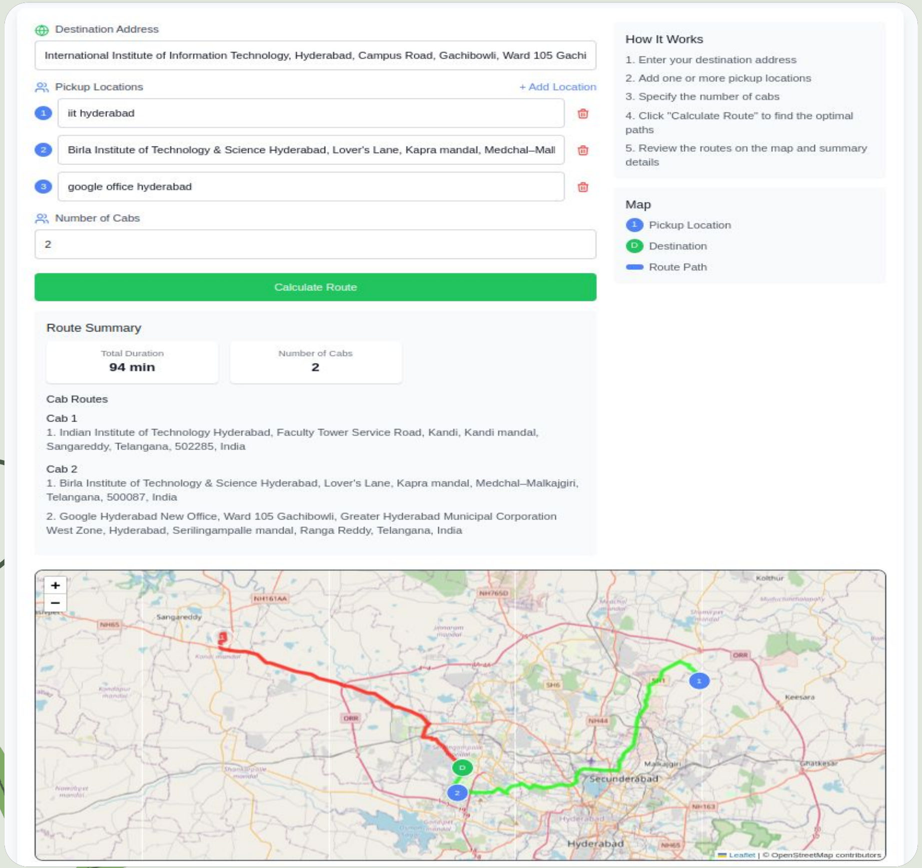
Routing and Optimisation



Can be categorised into 3 broad categories :

- **One-to-One** : Singular route from one source to one destination
- **Many-to-One** : Singular route from a source to destination; with multiple pick-up points in between
- **Many-to-Many** : Multiple routes (fixed number) covering a set of points. Each point falls under one of the route; but no route necessarily covers all points.

Routing and Optimisation



Can be categorised into 3 broad categories :

- **One-to-One** : Singular route from one source to one destination
- **Many-to-One** : Singular route from a source to destination; with multiple pick-up points in between
- **Many-to-Many** : Multiple routes (fixed number) covering a set of points. Each point falls under one of the route; but no route necessarily covers all points.

03

Challenges Faced

"Sometimes the smartest decision is knowing when to switch lanes."

Initial VROOM Hiccups

Before we moved to an API-based routing and optimisation approach, we tried hosting these algorithms locally on our machines. For this, we tried using VROOM (Vehicle Routing Open-source Optimization Machine). However, we faced some issues during the implementation, which ultimately led to the shift to AI-based routing.

Some of major challenges are as follows :

- **Setting Up**
  With its high dependency on other libraries like OSRM, necessitating the configuration and deployment of dedicated servers. This setup process proved to be intricate, requiring significant effort in managing dependencies and ensuring compatibility between different software components.

- **Huge Data Requirements**
  While the OSM data is relatively small (few MBs), it grows significantly upon converting to OSRM due to the locations, graphing and other metadata processing.

- **Computational Complexity**
  With increased data size, the computational requirements also scale; requiring very high compute as additional constraints for vehicle number, paths, etc. are introduced.

Issues with OSM & OSRM

As we progressed with the implementation and conducted related testing, we discovered significant limitations in OSM's **data quality** .

- Unlike the commonly used Google products, OSM's data library look-up aren't that advanced and optimised to infer and process any string combination to the correct address.
- For instance, look-ups for "BITS Hyderabad", "BITS Pilani, Hyderabad Campus", or even "Birla Institute of Technology & Science Pilani, Hyderabad Campus" (official / google term) would not yield any result.
- However, when you search "Birla Institute of Technology & Science Hyderabad", you get the desired result.

Additionally, we observed that OSRM's **response time and latency** were significantly slower compared to competing solutions, impacting overall performance and efficiency.

- Being an API call, at the end of the day, there is a latency aspect to the services.

This has raised concerns about its suitability for our use case, particularly in handling real-world address searches.

04

Future Plans

“An optimized future starts with a well-planned route today.”

Towards Google Maps API?

Following detailed discussions with the client, about possible solution, alternatives or work arounds, the client has asked us to try out **Google Maps API** for a more accurate and reliable mapping solution.

Several factors led us to reviewing this additional approach :

- **Superior Address Recognition** : Unlike OSM, Google Maps API successfully handles precise address searches, improving routing accuracy.
- **Robust Routing and Optimization** : Provides real-time traffic data and optimized routing based on multiple factors.
- **Ease of Integration** : Well-documented APIs with seamless implementation options.

There would be some challenges / trade-offs in terms of costs (outside of the free trial version, we are expected to work with) and limited customisations.

Note that this is not yet a confirmed shift. We are still awaiting the API keys and credentials, post which would perform various tests to finalise, with consultation of the client, our final implementation.

Watch Out in R2 For

Fleet Management

Implementing constraints handling for passenger capacity, timing preferences to attain the final goal of employee transportation

User-specific Features

Implementing login and user levels for accessing / restricting various features based on user level (Admin, Employee, Driver)

Improvements Beyond

Local Server

The current implementation makes use of online-hosted servers to get the optimised routes. As a service provider, one would like to host such services in-house. Reducing dependency and allowing customisations.

Real-Time Updates

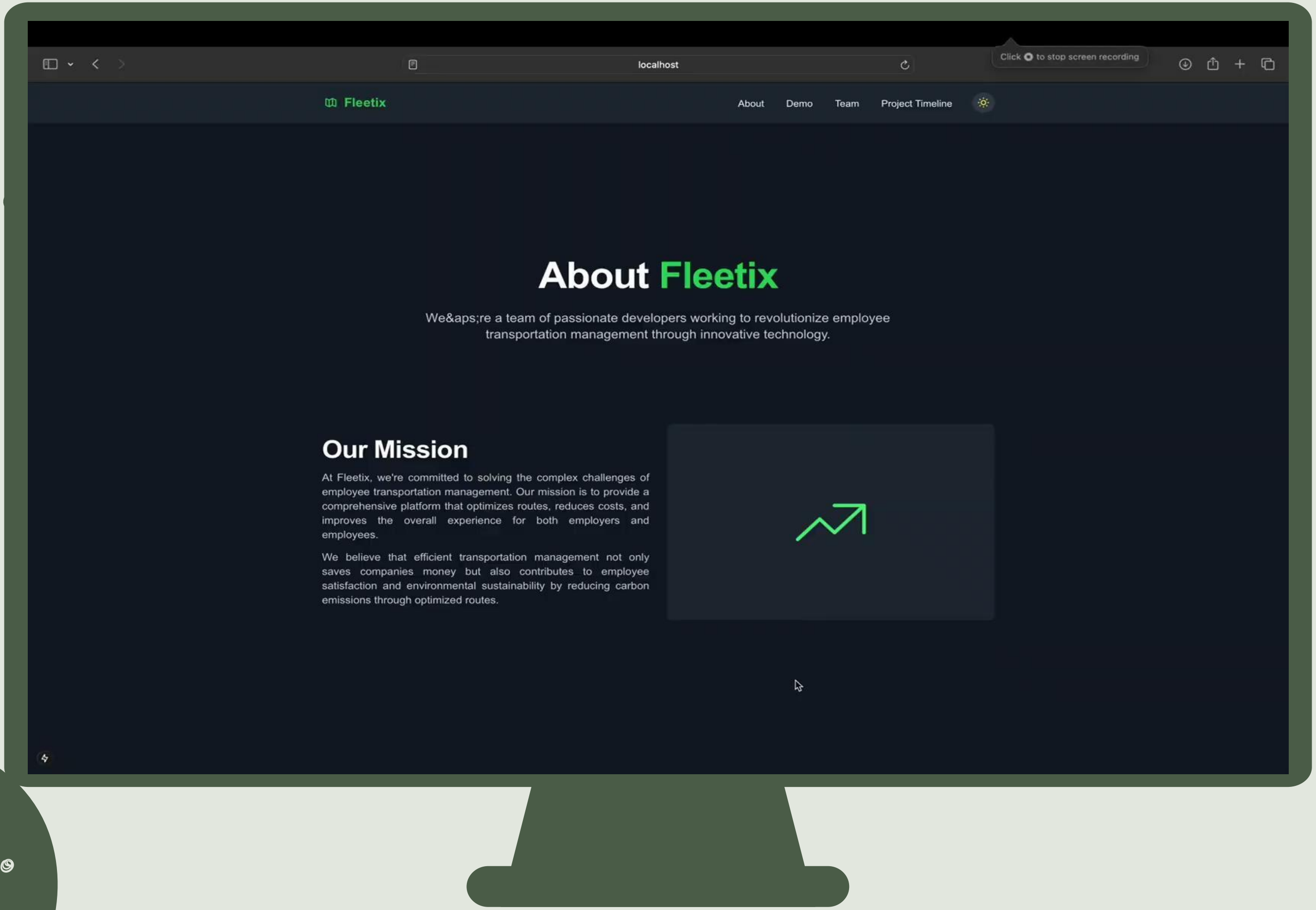
Allow for real-time route updation for status tracking of a particular vehicle, employee and driver. While OSRM doesn't support this feature, we are looking into other solutions like Google Maps API, etc.

Note that if time permits, we would try to implement these features by R2!

05

Demonstration

“A demonstration should not just show features - it should reveal possibilities.”



SCAN ME

Scan for Website!

06

Our Learnings

“Progress isn’t about avoiding mistakes, but about learning from them.”

Learnings

Technical

1. **Development Technologies**
   - Next.js & TypeScript for frontend development
   - Deployment using Vercel

2. **Geospatial Technologies**
   - OSM for map data and visualization
   - OSRM for route calculation and optimization
   - Nominatim for geocoding and address processing

Non-Technical

1. **Tackling Project Structure Challenges**
   - The non-modular nature of the project meant tasks were highly interdependent and required holistic understanding of the entire system
2. **Communication**
   - Critical for synchronising work on interconnected components and Team alignment on technical approach became essential.
3. **Work Management**
   - Handling the Difficulty in parallelising work efficiently due to dependencies

Work Distribution

Member	Main Role	Activities
Akshitha	Routing & Optimisation	Project Plan & Updated Plan Document, Research on Geocoding, Implement route algorithms (Many-to-One & Many-to-Many), Implement Website Route Pages, UML Diagrams (for SRS Document), Sequence Diagrams (for Design Document)
Aryanil	Research & VROOM	Git Repository setup, Team MoM Writeup, Research on TSP Algorithm (including initial implementation), Boilerplate for VROOM implementation, Implement Website Teams Page, Class Diagrams (for Design Document)
Aviral	Coordination & Frontend	Coordinate Meetings, Software Requirement Specification, Research on Heuristics & Grasshopper Optimization, Initialise Codebase, Design Document (Design Rationale), Implement Geocoding, Website Home, Navbar, Demo and Timeline Pages
Satvik	Documentation (Status Tracker and MoMs) & Backend	Status Tracker updates, Team MoM Writeups, Research on PgRouting, VROOM Implementation, Database Setup, Backend Setup and Login Page Development
Sherley	Market Research & Documentation (Diagrams and Synopsis)	Market Research on Existing Technology, Project Synopsis Writeup, Client Meeting MoM Writeup, UML Diagrams & Sequence Diagrams for Documentation, Code Function Blocks Implementation, Record Demo Video, Implement Footer and Suggestion Boxes

Acknowledgments

Lastly, we would like to extend our gratitude to :

- **Professor Raghu Reddy** for providing us with the opportunity to work on this project.
- **Our Client, Akhil** for his continuous coordination, constructive feedback, and support in shaping the project.
- **Teaching Assistant, Ashna Dua** for her guidance and assistance at every stage of the project.

Thank You!

Thank You!

Any Questions, Feedbacks, Suggestions would be appreciated!

- Team 28 -

Akshitha, Aryanil, Aviral, Satvik, Sherley